UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

UNIVERSIDAD ESTATAL AMAZÓNICA

TECNOLOGÍAS DE LA INFORMACION

ASIGNATURA

ESTRUCTURA DE DATOS

SEMESTRE

TERCERO “D”

ESTUDIANTE

CARLOS JEFFERSON CUEVA SANCHEZ

DOCENTE

ING. SANTIAGO ISRAEL NARVAEZ GUERRERO

ENLACE

https://github.com/CJCuevas1/Arbol_Binario_Semana_14/tree/main/ProyectoBST

Introducción

El presente proyecto detalla la implementación de un **Árbol Binario de Búsqueda (BST)** utilizando el lenguaje de programación **C#**. El objetivo es demostrar el manejo de estructuras de datos dinámicas mediante la creación de nodos que almacenan valores enteros y referencias a subárboles. A través de una interfaz de consola, el sistema permite realizar operaciones fundamentales como la inserción, búsqueda y eliminación, así como diversos tipos de recorridos para verificar la organización de la información

Capturas de pantalla

Clase Principal (Program.cs): Muestra el punto de entrada del programa y la implementación de un menú interactivo mediante un ciclo do-while, permitiendo al usuario gestionar el árbol de forma dinámica

Program.cs × ArbolBinarioBusqueda.cs Nodo.cs

Program.cs > { } ProyectoBST > Program > Main(string[]) : void

```
namespace ProyectoBST
{
    0 referencias
    internal class Program
    {
        0 referencias
        static void Main(string[] args)
        {
            ArbolBinarioBusqueda arbol = new ArbolBinarioBusqueda();
            int opcion;

            do
            {
                Console.Clear();
                Console.WriteLine("===== ÁRBOL BINARIO DE BÚSQUEDA (BST) =====");
                Console.WriteLine("1. Insertar valor");
                Console.WriteLine("2. Buscar valor");
                Console.WriteLine("3. Eliminar valor");
                Console.WriteLine("4. Mostrar recorrido Preorden");
                Console.WriteLine("5. Mostrar recorrido Inorden");
                Console.WriteLine("6. Mostrar recorrido Postorden");
                Console.WriteLine("7. Mostrar valor mínimo");
                Console.WriteLine("8. Mostrar valor máximo");
                Console.WriteLine("9. Mostrar altura del árbol");
                Console.WriteLine("10. Limpiar árbol");
                Console.WriteLine("11. Salir");
                Console.Write("Seleccione una opción: ");

                if (!int.TryParse(Console.ReadLine(), out opcion))
                {
                    Console.WriteLine("Opción inválida.");
                    Pausa();
                    continue;
                }
```

```
ArbolBinarioBusqueda.cs > { } ProyectoBST > ArbolBinarioBusqueda > Limpiar() : void
using System;
using System.Collections.Generic;

namespace ProyectoBST
{
    9 referencias
    public class ArbolBinarioBusqueda
    {
        16 referencias
        private Nodo raiz;

        1 referencia
        public ArbolBinarioBusqueda()    El elemento campo "raiz" que no acepta v
        {
            raiz = null;    No se puede convertir un literal NULL en un tipo de r
        }

        // INSERTAR
        1 referencia
        public void Insertar(int valor)
        {
            raiz = InsertarRecursivo(raiz, valor);
        }

        3 referencias
        private Nodo InsertarRecursivo(Nodo actual, int valor)
        {
            if (actual == null)
                return new Nodo(valor);

            if (valor < actual.Valor)
                actual.Izquierdo = InsertarRecursivo(actual.Izquierdo, valor);
            else if (valor > actual.Valor)
                actual.Derecho = InsertarRecursivo(actual.Derecho, valor);
            else
```

Lógica del Árbol (ArbolBinarioBusqueda.cs): Presenta la inicialización de la raíz y el método de inserción recursiva, el cual organiza los elementos menores a la izquierda y los mayores a la derecha.

```
Nodo.cs > { } ProyectoBST
namespace ProyectoBST
{
    18 referencias
    public class Nodo
    {
        15 referencias
        public int Valor { get; set; }
        15 referencias
        public Nodo Izquierdo { get; set; }
        18 referencias
        public Nodo Derecho { get; set; }

        1 referencia
        public Nodo(int valor)    El elemento propiedad "Izquierdo" que no acept
        {
            Valor = valor;
            Izquierdo = null;    No se puede convertir un literal NULL en un tip
            Derecho = null;    No se puede convertir un literal NULL en un tipo
        }
    }
}
```

Definición del Nodo (Nodo.cs): Se observa la estructura base del árbol, definiendo las propiedades para el valor entero y los punteros de referencia hacia los hijos Izquierdo y Derecho

Ejecución del Código

```
Nodo.cs > { } ProyectoBST
namespace ProyectoBST
{
    18 referencias
    public class Nodo
    {
        15 referencias
        public int Valor { get; set; }
        15 referencias
        public Nodo Izquierdo { get; set; }
        18 referencias
        public Nodo Derecho { get; set; }

        1 referencia
        public Nodo(int valor)    El elemento propiedad "Izquierdo" que no acept
        {
            Valor = valor;
            Izquierdo = null;    No se puede convertir un literal NULL en un tip
            Derecho = null;    No se puede convertir un literal NULL en un tipo
        }
    }
}
```

```
===== ÁRBOL BINARIO DE BÚSQUEDA (BST) =====
1. Insertar valor
2. Buscar valor
3. Eliminar valor
4. Mostrar recorrido Preorden
5. Mostrar recorrido Inorden
6. Mostrar recorrido Postorden
7. Mostrar valor mínimo
8. Mostrar valor máximo
9. Mostrar altura del árbol
10. Limpiar árbol
11. Salir
Seleccione una opción: 1
Ingrese un valor entero para insertar: 60
```

Menú de Opciones: Interfaz de usuario en consola que lista las 11 funciones disponibles, desde la inserción y búsqueda hasta el cálculo de la altura del árbol

```
===== ÁRBOL BINARIO DE BÚSQUEDA (BST) =====
1. Insertar valor
2. Buscar valor
3. Eliminar valor
4. Mostrar recorrido Preorden
5. Mostrar recorrido Inorden
6. Mostrar recorrido Postorden
7. Mostrar valor mínimo
8. Mostrar valor máximo
9. Mostrar altura del árbol
10. Limpiar árbol
11. Salir
Seleccione una opción: 1
Ingrese un valor entero para insertar: 60
```

Inserción de Datos: Ejemplo práctico donde se ingresa el valor "60" como nodo inicial para comenzar a construir la estructura del BST

```
===== ÁRBOL BINARIO DE BÚSQUEDA (BST) =====
1. Insertar valor
2. Buscar valor
3. Eliminar valor
4. Mostrar recorrido Preorden
5. Mostrar recorrido Inorden
6. Mostrar recorrido Postorden
7. Mostrar valor mínimo
8. Mostrar valor máximo
9. Mostrar altura del árbol
10. Limpiar árbol
11. Salir
Seleccione una opción: 4
Recorrido Preorden:
60 - 20 - 50
```

Recorridos del Árbol: Demostración de los recorridos **Preorden** (60 - 20 - 50) e **Inorden** (20 - 50 - 60), este último confirmando la ordenación ascendente de los datos

Recorrido Inorden

```
===== ÁRBOL BINARIO DE BÚSQUEDA (BST) =====
1. Insertar valor
2. Buscar valor
3. Eliminar valor
4. Mostrar recorrido Preorden
5. Mostrar recorrido Inorden
6. Mostrar recorrido Postorden
7. Mostrar valor mínimo
8. Mostrar valor máximo
9. Mostrar altura del árbol
10. Limpiar árbol
11. Salir
Seleccione una opción: 5
Recorrido Inorden:
20 - 50 - 60
```

```
===== ÁRBOL BINARIO DE BÚSQUEDA (BST) =====
1. Insertar valor
2. Buscar valor
3. Eliminar valor
4. Mostrar recorrido Preorden
5. Mostrar recorrido Inorden
6. Mostrar recorrido Postorden
7. Mostrar valor mínimo
8. Mostrar valor máximo
9. Mostrar altura del árbol
10. Limpiar árbol
11. Salir
Seleccione una opción: 7
El valor mínimo es: 20
```

```
===== ÁRBOL BINARIO DE BÚSQUEDA (BST) =====
1. Insertar valor
2. Buscar valor
3. Eliminar valor
4. Mostrar recorrido Preorden
5. Mostrar recorrido Inorden
6. Mostrar recorrido Postorden
7. Mostrar valor mínimo
8. Mostrar valor máximo
9. Mostrar altura del árbol
10. Limpiar árbol
11. Salir
Seleccione una opción: 9
La altura del árbol es: 2
```

Métricas y Resultados: Visualización de funciones auxiliares que identifican correctamente el valor mínimo (20) y la altura total del árbol (2) tras las inserciones realizadas

Conclusión

La ejecución del programa confirma el correcto funcionamiento de la lógica recursiva aplicada en la estructura del árbol. Se validó mediante pruebas en consola que el recorrido **Inorden** presenta los datos de forma ascendente (20 - 50 - 60), lo cual es la característica principal de un BST. Además, las funciones auxiliares para determinar el valor mínimo y la altura del árbol arrojaron resultados precisos según los datos ingresados, cumpliendo así con los requerimientos técnicos del sistema